Proposal: Develop and Integrate Bitcoin-Only Firmware for Cypherock X1 Hardware Wallet

Name and Contact Information

- **Name**: Utkarsh Maurya
- **Email**: projects.utkarshMaurya@gmail.com
- **Discord**: master.utkarsh
- **University Name**: National Institute of Technology, Hamirpur
- **Country**: India
- **GitHub**: @pro-utkarshM
- **LinkedIn**: @utkarsh-maurya-connect

Synopsis

This project aims to develop a Bitcoin-only firmware for the Cypherock X1 hardware wallet. By eliminating support for altcoins, the firmware will optimize security, reduce attack vectors, and streamline performance for Bitcoin transactions. The firmware will employ industry-standard cryptography (e.g., **ECDSA** on **secp256k1**) and adhere to Bitcoin standards (BIP-32/39/44) while ensuring seamless integration with Bitcoin Core nodes and modern Bitcoin wallets. The project will contribute to an enriched hardware wallet ecosystem and enhance interoperability within the broader Bitcoin network.

Project Objectives and Ecosystem Integration

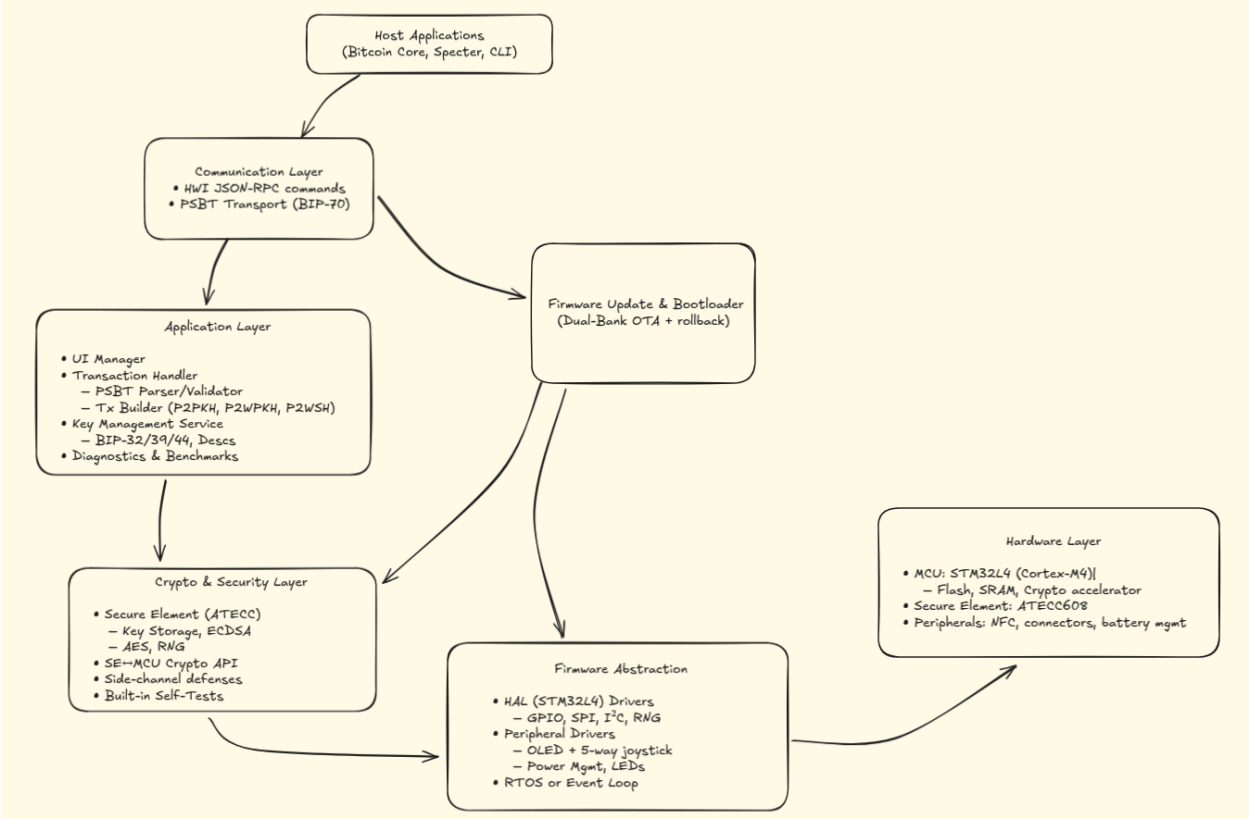
- **Bitcoin-Specific Functionality**: Deliver secure key management, transaction signing, and UTXO handling exclusively optimized for Bitcoin.

- **Ecosystem Integration**:

    - **Interoperability**: Ensure that the firmware communicates flawlessly with Bitcoin Core nodes and popular SPV wallets (e.g., Electrum, Wasabi), making the device a trusted component in the Bitcoin ecosystem.
    - **Standards Compliance**: Fully support Bitcoin Improvement Proposals (BIPs) like BIP-32 (HD wallets), BIP-39 (mnemonic phrases), and BIP-44 (coin type derivation), ensuring compatibility with current and future Bitcoin wallet applications.

- **Open-Source Synergy**: Facilitate collaboration with existing open-source projects, serving as both a reference design and a complementary solution to existing hardware wallets in the Bitcoin ecosystem.

Proposed Architecture (Github)



Project Plan

Phase 1: Initial Setup and Research (Weeks 1-3)

- **SDK and Hardware Analysis**:
  - Deep-dive into the Cypherock X1 SDK, firmware architecture, communication interfaces (USB, BLE), and memory constraints.
  - Review hardware documentation to understand clock speeds, available memory, secure element features, and power consumption profiles.

- **Cryptographic and Protocol Review**:

    - Investigate the current cryptographic implementations (e.g., secure RNG, key generation modules) and assess how they match with Bitcoin's requirements.
    - Map out how the existing firmware handles multi-currency support to pinpoint aspects to remove or optimize.

- **Integration Blueprint**:

    - Define how the firmware will integrate with Bitcoin networks; design API endpoints or communication protocols to interact with Bitcoin Core nodes and SPV wallet standards.
    - Identify potential interoperability challenges and plan to leverage existing libraries (e.g., **libbitcoin**, Bitcoin Core libraries) that are proven in production.

Phase 2: Bitcoin-Only Firmware Development (Weeks 4-7)

- **Key Management and HD Wallet Implementation**:

    - Implement BIP-32 HD wallet functionality for deterministic key derivation, along with BIP-39 mnemonic integration for recovery.
    - Use hardware-backed secure storage techniques, leveraging the Cypherock X1 secure element if available.

- **Transaction Signing Engine**:

    - Develop a signing engine focused solely on Bitcoin transactions, using **ECDSA** over the secp256k1 curve.
    - Validate that the transaction signing process supports legacy (P2PKH), P2SH, and SegWit formats (P2WPKH/P2WSH).

- **Strip Altcoin Code & Optimize**:

    - Eliminate all non-Bitcoin libraries and code paths, refactoring the firmware to reduce memory usage and execution cycles.

- **Library Integration & Benchmarking**:

    - Integrate established Bitcoin libraries for transaction construction and serialization.
    - Establish preliminary performance benchmarks during development to ensure the firmware meets latency and throughput targets for rapid transaction signing.

Phase 3: Security and Performance Enhancement (Weeks 8-9)

- **Comprehensive Security Audits**:
    - Perform static and dynamic code analysis to identify potential vulnerabilities (e.g., side-channel leakage, buffer overflows).
    - Implement tamper detection routines and secure memory management practices (e.g., overwriting sensitive buffers after use, memory encryption if supported).
    - Validate the quality of the RNG, comparing against industry benchmarks and standards.

- **Risk Assessment and Mitigation**:
    - **Hardware Limitations**: Identify and document potential issues (e.g., limited RAM, CPU constraints) with fallback strategies, such as modular code design and prioritization of critical operations.
    - **Platform Compatibility**: Test against multiple configurations of Bitcoin software (different Bitcoin node versions and wallet APIs) to ensure interoperability.
    - **Fallback and Recovery**: Design an emergency rollback mechanism to revert firmware updates in case of malfunction or detected security breaches.

- **Performance Optimization**:
    - Profile transaction signing routines, iteratively refining code to maximize performance under resource-constrained conditions.
    - Use hardware benchmarks and compare against open-source wallet performance metrics to validate improvements.

Phase 4: Integration Testing, Documentation, and Final Release (Weeks 10-12)

- **Integration Testing with Bitcoin Ecosystem**:
    - Test firmware interoperability with Bitcoin Core nodes, SPV wallets, and third-party Bitcoin applications.
    - Simulate real-world scenarios on the Bitcoin testnet to verify key management, transaction validity, and wallet communication.

- **Documentation and User Guides**:
    - Create comprehensive technical documentation covering firmware architecture, cryptographic implementations, and integration protocols.

    - Develop user guides detailing installation, key backup procedures (mnemonics), and troubleshooting steps.

- **Final QA and Release Preparation**:

    - Conduct a final round of bug fixes, performance tuning, and security re-evaluation.
    - Package the firmware for distribution with clear versioning and update protocols.

Challenges, Risks, and Mitigation Strategies

1. **Hardware Constraints**:

    - *Challenge*: Limited processing power and memory on the Cypherock X1 may restrict advanced features.
    - *Mitigation*: Optimize code for low memory footprint and use profiling to prioritize critical functions; modularize code to allow future upgrades.
2. **Platform Compatibility**:

    - *Challenge*: Ensuring seamless interoperability with various Bitcoin nodes and wallet clients.
    - *Mitigation*: Leverage proven open-source libraries and standards (BIPs) and perform extensive integration testing with multiple platforms.
3. **Security Vulnerabilities**:

    - *Challenge*: Hardware wallets are prime targets; any weakness could lead to catastrophic key compromise.
    - *Mitigation*: Adhere to strict cryptographic practices, use hardware-backed security features, perform both static and dynamic vulnerability assessments, and implement redundant security measures such as secure key wiping and memory isolation.

4. **Ecosystem Integration**:

    - *Challenge*: Demonstrating that this firmware is not an isolated project but a valuable part of the Bitcoin ecosystem.
    - *Mitigation*: Develop open APIs for integration with widely used Bitcoin clients and nodes, contribute benchmarks and performance comparisons, and collaborate with existing open-source Bitcoin projects to encourage adoption.

Deliverables

1. **Bitcoin-Only Firmware**: A fully operational firmware version for the Cypherock X1 with dedicated Bitcoin functionalities.

2. **Comprehensive Test Reports**: Detailed unit, integration, and security test results, including performance benchmarks.

3. **Complete Documentation**: Technical documentation covering the firmware's design, risk mitigations, integration strategies, and user manuals.

4. **Release Package**: A version-controlled, release-ready firmware package complete with update and rollback protocols.

Benefits to the Community

- **Focused Security**: By concentrating solely on Bitcoin, the firmware minimizes attack vectors, significantly enhancing security over multi-currency firmware.

- **Optimized Performance**: Reduced complexity results in faster transaction signing and improved user experiences, directly benefiting Bitcoin end-users.

- **Ecosystem Synergy**: The project establishes clear integration pathways with existing Bitcoin infrastructures, promoting higher adoption and collaborative enhancements.

- **Open-Source Impact**: The contribution will serve as a reference implementation for Bitcoin-only hardware wallets and drive innovation within the open-source Bitcoin community.

Biographical Information

I am a passionate embedded systems developer specializing in cryptographic applications and hardware security. Currently pursuing a degree in Electronics and Communication Engineering at NIT Hamirpur, I bring a strong foundation in C/C++ programming, systems-level development, and secure software-hardware integration within the cryptocurrency space.

My technical focus includes secure firmware development, Bitcoin transaction protocols (particularly PSBT), and building reliable host-device interfaces. I've contributed to open-source blockchain initiatives and have hands-on experience working with Bitcoin Core's JSON-RPC API, transaction serialization formats, and SPV clients like Electrum.

For this project, I aim to build robust CLI tooling that interfaces with the Cypherock X1 Vault, enabling seamless PSBT-based transaction signing via USB/NFC and secure shard management. I'm particularly interested in ensuring compatibility with Bitcoin Core and maintaining a high degree of modularity in the toolchain design.

To deepen my understanding, I am currently studying the PSBT format in detail, the design of hardware wallets, and secure elements used for multisig vaults. I'm also analyzing how the X1's firmware handles CRI interfaces and encrypted shard synchronization to ensure the host-side logic integrates smoothly.

My prior projects demonstrate a solid track record in systems programming and hardware-oriented development:

- myOwnOS: A minimal operating system built from scratch to explore bootloader design, memory management, and kernel-user separation.

- baking-pi-3: A modified version of the Baking Pi course, ported for Raspberry Pi 3B, including UART debugging and peripheral control.

- **Router Firmware CVE Work**: Reverse-engineered router firmware to identify vulnerabilities and study embedded OS behavior.

- **Coreboot Experiments**: Flashed laptops with Coreboot to enhance system transparency and security through open-source firmware.

These experiences have given me the practical insight needed to contribute to the Cypherock ecosystem and advance the usability and security of Bitcoin signing workflows through hardened, open tooling.

Conclusion

By developing a Bitcoin-only firmware for the Cypherock X1, this project aims to deliver a secure, high-performance, and ecosystem-integrated solution specifically tailored for Bitcoin users. Addressing hardware limitations, security vulnerabilities, and integration challenges

head-on, this firmware will not only enhance the user experience but also solidify its role as a critical component within the broader Bitcoin ecosystem.